

# Wikis and Collaborative Systems for Large Formal Mathematics

Cezary Kaliszyk<sup>1</sup> and Josef Urban<sup>2(✉)</sup>

<sup>1</sup> University of Innsbruck, Innsbruck, Austria

<sup>2</sup> Czech Technical University in Prague, Prague, Czech Republic  
josef.urban@gmail.com

**Abstract.** In the recent years, there have been significant advances in formalization of mathematics, involving a number of large-scale formalization projects. This naturally poses a number of interesting problems concerning how should humans and machines collaborate on such deeply semantic and computer-assisted projects. In this paper we provide an overview of the wikis and web-based systems for such collaboration involving humans and also AI systems over the large corpora of fully formal mathematical knowledge.

## 1 Introduction: Formal Mathematics and Its Collaborative Aspects

In the last two decades, large corpora of complex mathematical knowledge have been encoded in a form that is fully understandable to computers [8, 15, 16, 19, 33, 41]. This means that the mathematical definitions, theorems, proofs and theories are explained and formally encoded in complete detail, allowing the computers to *fully understand the semantics* of such complicated objects. While in domains that deal with the real world rather than with the abstract world researchers might discuss what *fully formal encoding* exactly means, in computer mathematics there is one undisputed definition. A fully formal encoding is an encoding that ultimately allows computers to verify to the smallest detail the correctness of each step of the proofs written in the given logical formalism. The process of writing such computer-understandable and verifiable theorems, definitions, proofs and theories is called *Formalization of Mathematics* and more generally *Interactive Theorem Proving* (ITP).

The ITP field has a long history dating back to 1960s [21], being officially founded in 1967 by the mathematician N.G. de Bruijn, with his work on the Automath system [13]. The development of a number of approaches (LCF, NQTHM, Mizar) followed in the 1970-s, resulting in a number of today's established ITP systems such as HOL (Light) [20], Isabelle [61], Mizar [17], Coq [11],

---

C. Kaliszyk—Supported by the Austrian Science Fund (FWF): P26201.

J. Urban—Supported by NWO grant nr. 612.001.208 and ERC Consolidator grant nr. 649043 *AI4REASON*.

ACL2 [32] and PVS [46]. This development was often intertwined with the development of its cousin field of *Automated Theorem Proving* [51] (ATP), where proofs of conjectures are attempted fully automatically, without any human assistance. But unlike ATP systems, the ITP systems allow human-assisted formal theory encoding and proving of theorems that are often beyond the capabilities of the fully automated systems.

Twenty years ago, the anonymous QED Manifesto [1] proposed to start a unified encyclopedic effort formalizing all of today’s mathematics. This led to a lot of discussion about suitable logical foundations for such a unified effort, and to two QED workshops in 1995 and 1996 [42]. While no such unified foundations and a top-down managed effort emerged, formal libraries of impressive size have been built since then with ITPs: the Mizar Mathematical Library (MML) contains today over 50000 theorems and the Isabelle/HOL and the Flyspeck project (with the large HOL Light libraries) have some 20000 theorems each. ITP is also becoming an indispensable technology for verifying complex software-assisted proofs, and other complicated (e.g., hardware and software) designs. Other exact sciences, such as economics [39] and physics [2] have started to be fully formally encoded recently.

Recent examples of the large projects in formal mathematics include the completed formal proofs of the Kepler conjecture (Flyspeck) [19], the Odd Order theorem [16], the Four Color theorem [15], and verification of more than a half of the Compendium of Continuous Lattices textbook [8]. Verification of the seL4 kernel [33] and the CompCert compiler [41] show comparable progress in full-scale verification of complicated software. Such projects are often linked to various advances in verification technology, such as strong automation methods [27, 28, 35] that allow less and less verbose formal proofs, however, ITP still remains very labor-intensive. For example, the Flyspeck project is estimated to take 20 person-years, and the Feit-Thompson project took about twice as much.

This means that such large projects in formal mathematics have also very important collaboration aspects. Sometimes they may be managed in a very tight top-down manner, with the formal terminology, naming conventions and majority of stated lemmas being designed by the project leaders, and only the proofs are written by other team members. This has many advantages in consistency, similar for example to the one-man (or one-committee) design of the “upper ontologies” in common-sense reasoning projects such as SUMO [45] and CYC [49]. Sometimes such total top-down control is however missing, and the projects or libraries grow much more chaotically, similar to the way how Wikipedia grows, i.e., only with a basic set of rules for contributing and a lot of later (possibly collaborative) refactoring of the formal terminology, statements, proofs, and theories.

The combination of all these aspects makes formal mathematics a very interesting domain for several reasons:

1. It allows collaboration of experts from very different mathematical areas, both professionals and students and amateurs. As long as the formal proofs are correct, nobody needs to be interested about who wrote them and how,

and if they understood the defined concepts correctly. All this consistency and proof checking is ensured by the underlying ITP systems. This *power of objectivity* has been noted by John McCarthy who proposed to transfer such mechanisms to other sciences and venues of lives.<sup>1</sup>

2. In a similar way, it allows collaboration with machines, particularly AI and ATP systems. Such systems can use very strong semantic search methods such as automated theorem proving, to assist the humans. Again, the final correctness and consistency of such strong machine assistance is checked by the ITP systems.
3. It has all kinds of relations to less formal domains. A very interesting topic is how to present the formal proofs to humans, and also how to assist humans as much as possible with converting informal mathematics into formal. The latter includes all kinds of specialized editors and authoring environments. Such editors and environments may support various semantic features, can be web-based, allow hyperlinking and inter-linking via Semantic Web techniques to more fuzzy concepts defined in Wikipedia and DBpedia, etc. A related interesting topic is how to connect and re-use the formalizations done in various formal systems, or using various libraries.

The rest of this paper is organized as follows. Section 2 summarizes the main differences between encyclopedic efforts like Wikipedia and the reality of formal mathematics. In Sect. 3 we introduce formal mathematical wikis and their main components, such as formal verification, suitable rendering, versioning, editing, and semantic assistance. Section 4 then discusses the examples of today's formal wikis and their various features and subsystems.

## 2 Some Obstacles to Wikis for Formal Mathematics

In the informal world, Wikipedia can today be largely considered as *The World Encyclopedia* written in a collaborative wiki style, covering vast number of topics and serving more semantics projects such as DBpedia and Wikidata, which can in turn be considered to be the hub of the Semantic Web. In formal mathematics, there is unfortunately no such unique resource, despite the early QED proposals and efforts. Whether such fragmentation is necessary is not clear, however, the following factors largely contribute to this state.

*Disagreement about the logical and mathematical foundations.* Mathematicians and logicians have different opinions about what should be the most convenient and the most correct foundations for mathematics, ranging from philosophical topics such as constructive versus classical logic, to more practical topics such as using higher-order logic (HOL) versus set theory versus type theory. There are logical frameworks (such as Isabelle [61] or Twelf [52]) that try to cater for everybody, however those may be viewed as an additional burden by some, and their universality, practicality and foundations may be further questioned. The most widely used systems today commit to just one foundational framework.

---

<sup>1</sup> <http://www-formal.stanford.edu/jmc/future/objectivity.html>.

*Disagreement about the formal language and its mechanisms.* This is very similar to the disagreement about the appropriateness of various programming languages for various projects. Analogously to that, the chance of all formalizers ultimately agreeing on some of the current formal languages is quite small.

*Disagreement about how formal concepts should be defined and mathematics should be built.* Even when within common foundations and when using a common formal language and an ITP system, there are sometimes different approaches to formalizing mathematics. In the Coq system, there are several definitions of real numbers, algebraic hierarchies (monoids, groups, rings, fields, modules, etc.) have been built in several competing ways, etc. In the Mizar Mathematical Library, there are several definitions of graphs, categories, and other structures, and parallel theorems about these parallel concepts. Again, this is quite analogous to the current state in programming, where several programming libraries/projects may address the same task differently, according to the tastes and opinions of its implementors. Unlike in Wikipedia, one usually does not prefer to present the different implementations side-by-side within one library, because that makes the library less focused and compact, possibly leading the library's users to confusion and multiple efforts in the same direction.

All these issues have considerable impact on the kind of collaborative tools that can be useful for formalization of mathematics. Around 2004, inspired by the success of Wikipedia, various proposals started to appear suggesting to speed up the relatively slow formalization efforts by wiki-like methods.<sup>2</sup> Ten years later, it is however still not completely clear what is the right combination of features suitable for fast collaborative formalization. We discuss the main components in the next section. While a number of interesting wiki systems have been produced for less formal mathematics, knowledge engineering, research support, and related domains [9, 40, 44, 62], so far there has been no successful attempt to port such systems to the fully formal setting.

### 3 Formal Wikis and Their Main Components

Traditionally, formal mathematics used to be constructed locally by a single author, either in an editor or in a toplevel<sup>3</sup> of an interactive theorem prover. The prover immediately checks the steps, using its proof rules and the previous knowledge available in the formal libraries, which are locally installed. After the formal definitions, theorems and proofs are written in a file, the file is typically checked again in batch mode by the prover and included in the locally installed library. Various methods have evolved for making one's results available to others. For a long time this would be just an e-mail to library managers, who would then distribute the new library version using FTP or WWW. In the

<sup>2</sup> <http://wiki.mizar.org/twiki/bin/view/Mizar/MizarWishlist>.

<sup>3</sup> Many proof assistants have been implemented inside programming language interpreters and inherit their toplevels.

last decade, a major step towards fast collaborative development was done by switching to version control systems (VCS), which have become also one of the building blocks of today's formal wikis.

In general, a formal wiki would typically address the following six components to various extent:

**Formal verification** on the server. This is the defining element of the formalization process. Each text that is submitted to a formal wiki will first be evaluated with respect to a particular *formal verifier* (i.e., a proof assistant, ITP) implementing the logic rules and checking the statements and proofs with respect to the logic and the available formal library. Various result statuses of the formal verification are possible. For example, the formal text might be correctly parsed (all concepts are known and used in a correct way), but not completely proof-checked (some proofs make logical inferences that the proof checker cannot understand, perhaps requiring to refine some steps). Or changes in one part of the library might invalidate another part of the library, putting the library as a whole into an inconsistent state. Depending on the results of the verification, the wiki might either completely reject the text, or accept it with some status, usually also updating its existing library with the text, and/or possibly performing some other actions such as branching the development automatically.

**Formal library** or libraries available on the server, used to supply mathematical terminology and theorems needed for verification of more advanced texts. This is one of the main ways in which interactive theorem proving differs from completely automated theorem proving, where all the information needed for proving a particular statement is typically included in one file. Such large formal libraries are typically stored efficiently in some pre-compiled fast-loadable format, and their parts are loaded into the interactive provers using various inclusion mechanisms. One of the major issues that need to be addressed in formal wikis is the updating, refactoring and maintenance of such libraries of formal concepts, theorems and proofs, their versioning and suitable rendering.

**Versioning.** Often one wants to see older versions of various files in the library, track the changes done by various users, or even try to work with a non-default version of the library. For example MediaWiki, Wikipedia's internal wiki engine, uses sequential numbering of files (independent of other files), and for every change, the id of the user that made the change is stored with the file. Viewing of differences and histories is possible, although more limited than in more advanced versioning systems. This simplicity can also be an advantage: Wikipedia does not include branches, tags, etc., and the casual user does not need to understand such features. On the other hand, more advanced version control systems operate with the concept of change sets (or commits), which seem more appropriate when several files need to be simultaneously modified (for example when renaming some function) in order for the library to be in a consistent state. If such library consistency is the main concern which is strictly enforced, the Wikipedia-style file-based version control is insufficient, and such advanced version control systems with simultaneous multi-file commits are a necessity.

**(Semantic) Rendering.** Many interactive provers include documentation generators that process raw prover input files and generate rendered output. The output of a documentation generator is usually HTML or PDF format, with HTML being of particular interest for wikis. Links between files and items are created, different conceptual elements of the prover input are colored in different color, and sometimes mathematical formulas are rendered in a graphical way. Since we are dealing with a very semantic domain, various sorts of information can be added to make the proofs more understandable by the readers, various dependencies can be explicitly shown, etc. While the authors of formal articles typically know how to read the raw formal texts, the HTML presentation (often including MathML) is one of the main aspects of formal wikis that make formal mathematics accessible to newcomers. One of the main reasons is the typical brevity of the formal language which is often optimized for writing rather than reading, and the ubiquitous overloading of mathematical symbols like  $+$ , whose particular meaning in a particular context might be very hard to decipher for a casual reader. For example, the large Mizar Mathematical library defines about 200 different uses of the symbol  $+$  (for example addition introduced for natural numbers and addition in a group with addition use this symbol). Suitable HTML rendering can link such symbols to their definitions, or even allow their immediate preview by mechanisms such as mouse-over tooltips. In a similar way, aligning with various less formal resources such as textbooks is possible, and also aligning with Semantic Web resources such as Wikipedia or DBpedia.

**Editing.** While not strictly necessary, it is very useful to have a server part that allows interactive editing of the formal text in a way that resembles local work. Formal proof editors typically offer many advanced semantic features, such as stepping through a proof with the interactive prover and rendering the resulting proof state. Having some of the most used features as a part of the web-based editor is likely to make immediate browser-based updates of the wiki much more attractive for casual readers.

**(Semantic) Assisting Tools.** Searching for suitable previously defined terminology and suitable theorems is one of the main activities when formalizing mathematics. The better this process is assisted, the more efficient are the authors of formal articles. As any user of today's web search engines knows, server-based technology allows much more sophisticated search tools that can make use of much more expensive processing steps than the user's machine, and that can also benefit from processing much more data than is available on the user's machine. A typical example would be suggesting hints for a new proof based on machine learning on the large libraries of previous proofs stored in the wikis, or running dozens of automated theorem provers on a particular interactively entered goal, making use of a large ("cloud-based") parallelization on the server.

## 4 Examples of Formal Wikis and Related Systems

The existing examples and prototypes of formal mathematical wikis (and related systems) can be broadly divided by whom they serve and what is their primary purpose.

#### 4.1 Targeted Formal Wikis: Mizar Wiki and Others

Wikis for a particular formal library or a formalization project are perhaps the most visible examples of a wiki-like formal-proof technology. The main example is the *Mizar wiki* prototype [3, 57]. It implements fast parallelized server verification, library update and versioning on the server, and HTML rendering for the Mizar system and its large formal library. The HTML rendering can produce all kinds of additional semantic information such as linking symbols to their definitions, showing of the proof state after each proof step, linking of theorems and concepts to Wikipedia and DBpedia, etc.

The verification and rendering are implemented just as hooks to an advanced distributed version control system (DVCS, `git` in this case). This allows the authors to work with the wiki locally, collaborate with others using just the DVCS features, and replicate and distribute the wiki very easily, just as if it was a standard software project. The distinguished Mizar wiki server receives updates from authorized users (as `git pushes`), using authorization systems such as `gitolite`<sup>4</sup>, and verifies the updates prior to including them into the main or other branch.

The verification against a particular branch of the large library is done using a smart copy-on-write filesystem such as ZFS or BTRFS, making it possible to keep many versions (possibly for many users) of the large library and compiled files on the server without significant space overhead and repetition of work and making the cloning and creation of new (possibly experimental) versions very cheap and fast. The fast cloning also serves for the verification process, which necessarily first has to update the existing library before the library gets formally re-verified. If the re-verification fails, resulting in the rejection of the library update, the new ZFS or BTRFS clone is simply deleted, and the library lives in its original state. If the particular update of a particular version of the library satisfies the verification policy (and the library is thus in some kind of a consistent state after the update), the library is also re-rendered and made available for browsing and further updates.

Updates can be done either using a web interface or by power-users via `git`. The updates from `git` can be used for multifile updates (such as library-scale symbol renaming) that would break the library consistency if done on a file-by-file basis. Rudimentary search tools exist for the Mizar wiki, however most of the advanced semantic search functions are not yet integrated with it, and instead run as separate tools produced for distinguished versions of the Mizar library. Similarly, only basic web editing is implemented.

Similar early prototype wiki has been built for Coq [3], and a number of formalization projects and libraries share today some of these features. For example the Coq Users' Contributions<sup>5</sup> are managed inside a VCS, automatically compiled and rendered, however the versioning and user submission are quite restricted. The Isabelle Archive of Formal Proofs<sup>6</sup> (AFP) allows authors

<sup>4</sup> <http://gitolite.com/>.

<sup>5</sup> <http://www.lix.polytechnique.fr/coq/pylons/contribs/index>.

<sup>6</sup> <http://afp.sourceforge.net/>.

to update their entries via a DVCS (mercurial) and also provides automated server-based checking, however the HTML rendering is very limited, and the users typically cannot update other entries. In this sense, the archive resembles an evolving online journal, rather than a massively collaborative wiki or a software project.

## 4.2 Wikis Focused on Editing Support: ProofWeb

To use a proof assistant, one needs to install some software. In the case of HOL Light one needs OCaml with a particular version of CamlP5 compiled with special flags (different than those used by popular Linux distributions), and HOL Light code itself, possibly with checkpointing software. To use an interface to access the prover, one needs the Emacs mode and one of the supported Emacs versions. The process described above is already complicated, not to mention other operating systems and architectures, or additional desirable patches and libraries, or less commonly used provers. The first web interface to the Coq system, LogiCoq [47], would not provide any support for editing: the whole buffer would be sent with standard HTTP request and refreshes the whole page.

ProofWeb [25] provides a web-interface to various proof assistants, that allows ProofGeneral-style [6] interaction. It implements a client-server architecture with a minimal lightweight client interpreted by the browser, a specialized HTTP server and background HTTP based communication between them. The key element of the architecture is the asynchronous DOM modification technique (sometimes referred to as AJAX - Asynchronous JavaScript and XML or Web application). The client part is stored on the server, and when the user accesses the interface page, it is downloaded by the browser, which is able to interpret it without any installation. The user of the interface, accessing it with the browser, does not need to do anything when a modification is done on the server. Every time the user accesses a prover, the version of the prover that is currently installed on the server is used. The user can access any of the provers installed on the server (ProofWeb supports Coq, Isabelle, Matita, Lego, Plastic, and has a minimal support for HOL Light).

The first wiki for Coq that integrated formal text editing with proof assistant feedback [12] was implemented as an extension of the MediaWiki engine, that used ProofWeb for editing pages and rendered the completed articles with Coqdoc in the viewing mode allowing L<sup>A</sup>T<sub>E</sub>X snippets. The convenience of a centralized proof assistant environment made it also appealing for teaching (Fig. 1). By defining special tactics for basic logical rules and proof tree rendering code [31] ProofWeb became a convenient tool for various computer courses and has been to date used in 49 courses at 12 universities [22].

A dedicated web interface has also been developed for Matita [5]. Apart from allowing Unicode input and syntax highlighting, it can better exploit the hypertextual document structure offered by the prover, by providing various annotations and active elements. Similarly the Clide [50] interface is a web-reimplementation of the Isabelle interface. It provides a document-oriented interaction with the prover, and allows collaborative editing of an Isabelle script: the edit operations performed by each user are immediately propagated to all.

|                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                           | <b>File</b> <b>Display</b> <b>Templates</b> <b>Backward</b> <b>Forward</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <pre>Require Import ProofWeb. Parameter p q r : Prop. Theorem example_1_13 : (p /\ q -&gt; r) -&gt; p -&gt; (q -&gt; r). Proof.    imp_i H1.   imp_i H2.   imp_i H3.   imp_e (p /\ q).   exact H1.   con_i.   exact H2.   exact H3.</pre> | <div>2 subgoals</div> <div> <math display="block">\begin{array}{l} H1 : p \wedge q \rightarrow r \\ H2 : p \\ H3 : q \\ \hline p \end{array}</math> </div> <div>subgoal 2 is:</div> <div> <math display="block">\begin{array}{c} \dots \quad \dots \\ \hline p \quad q \\ \hline [p \wedge q \rightarrow r]^{H1} \quad p \wedge q \quad \wedge i \\ \hline r \quad \rightarrow e \\ \hline q \rightarrow r \quad \rightarrow i[H3] \\ \hline p \rightarrow q \rightarrow r \quad \rightarrow i[H2] \\ \hline (p \wedge q \rightarrow r) \rightarrow p \rightarrow q \rightarrow r \quad \rightarrow i[H1] \end{array}</math> </div> |

**Fig. 1.** The ProofWeb interface editing a Coq proof script with a Gentzen-style proof tree.

### 4.3 Meta Wikis: Agora, Flyspeck Wiki, and Others

While it has so far turned out to be hard to make a one-for-all formalization system and a formal library, the efforts to make a unified interface to different corpora of existing formal mathematics have never stopped.

A recent example is the *Agora wiki* prototype by Tankink [54]. Here, the user combines informal narratives written in the Creole syntax with antiquotations that allow transclusion of formal texts from an arbitrary formal library that has been suitably annotated using the OMDoc ontology developed by Lange [37,38]. This ontology provides a wide supply of types of mathematical knowledge items, as well as types of *relations* between them, e.g. that a proof proves a theorem. It is a reimplementaion of the conceptual model of the OMDoc XML markup language [34] for the purpose of providing semantic Web applications with a vocabulary of structures of mathematical knowledge.

Regardless of the exact details of the formal systems involved, and their output, the annotation process generally yields HTML+RDFa, which uses the OMDoc ontology as a vocabulary. For example, if the formal document contains an HTML rendition of the Binomial Theorem, Agora expects the following result (where the prefix *oo:* has been bound to the URI of the OMDoc ontology<sup>7</sup>):

```
<span typeof="oo:Theorem" about="#BinomialTheorem">...</span>
<span typeof="oo:Proof"><span rel="oo:proves" resource="#BinomialTheorem" />
...</span>
```

<sup>7</sup> <http://omdoc.org/ontology#>.

The “...” in this listing represent the original HTML rendition of the formal text, possibly including the information that was used to infer the annotations now captured by the RDFa attributes. @about assigns a URI to the annotated resource; here, we use fragment identifiers within the HTML document.

The corresponding RDF annotation has been so far done for the Mizar, Flyspeck and Coq libraries. For instance in Mizar this was done as a part of the XSL transformation that creates HTML from the Mizar’s custom semantic XML format [55]. While the OMDoc ontology defines vocabulary that seems suitable also for many Mizar internal proof steps, the current Mizar implementation only annotates the main top-level Mizar items, together with the top-level proofs. Even with this limitation this has already resulted in about 160000 annotations exported from the whole MML. The existing Mizar HTML namespace was re-used for the names of the exported items, such that, for example, the Brouwer Fixed Point Theorem:<sup>8</sup>

```
:: $N Brouwer Fixed Point Theorem
theorem Th14:
  for r being non negative (real number), o being Point of TOP-REAL 2,
    f being continuous Function of Tdisk(o,r), Tdisk(o,r)
  holds f has_a_fixpoint
proof ...
```

gets annotated as<sup>9</sup>

```
<div about="#T14" typeof="oo:Theorem">
  <span rel="owl:sameAs"
    resource="http://dbpedia.org/resource/Brouwer_Fixed_Point_Theorem"/> ...
  <div about="#PF23" typeof="oo:Proof"><span rel="oo:proves" resource="#T14"/> ... </div>
</div>
```

Apart from the appropriate annotations of the theorem and its proof, an additional *owl:sameAs* link is produced to the DBpedia (Wikipedia) “Brouwer.Fixed.Point.Theorem” resource. Such links are produced for all Mizar theorems and concepts for which the author defined a long (typically well-known) name using the Mizar ::\$N pragma. Such pragmas provide a way for the users to link the formalizations to Wikipedia (DBpedia, ProofWiki, PlanetMath, etc.), and the links allow data consumers (like Agora) to automatically mesh together different (Mizar, Coq, etc.) formalizations using DBpedia as the common namespace.

A particular instantiation of Agora is the *Flyspeck wiki* prototype [53] used for aligning, cross-linking, and potential further joint refactoring of Hales’s informal Flyspeck book [18] about the proof of the Kepler Conjecture, and the corresponding formal Flyspeck development. This alignment takes advantage of the following annotated L<sup>A</sup>T<sub>E</sub>X form (developed by Hales in his book), which already cross-links informal objects to some of the formal counterparts (formally defined symbols and theorem names living in the formalization):

<sup>8</sup> <http://mizar.cs.ualberta.ca/~mptp/7.12.02.4.178.1142/html/brouwer.html#T14>.

<sup>9</sup> T14 is a *unique internal* Mizar identifier denoting the theorem. Th14 is a (possibly non-unique) *user-level* identifier (e.g., Brouwer or SK300 would result in T14 too).

```

\begin{definition}[polyhedron]\guid{QSRHLXB}
A \newterm{polyhedron} is the
intersection of a finite number of closed half-spaces in
 $\mathbb{R}^n$ .
\end{definition}

\begin{lemma}[Krein--Milman]\guid{MUGGQUF}
Every compact convex set  $P \subset \mathbb{R}^n$  is the convex hull
of its set of extreme points.
\end{lemma}

```

where QSRHLXB and MUGGQUF are the identifiers of the formal definition and theorem. The text contains many further mappings between informal and formal concepts, e.g.:

```

\formaldef{\op{azim}(x)}{azim\_fan}
\formaldef{M"obius contour}{is\_Moebius\_contour}
\formaldef{half space}{closed\_half\_space, open\_half\_space}

```

The wiki relies on MathJaX for rendering the rich informal mathematics, and on custom transformations from L<sup>A</sup>T<sub>E</sub>X to the Creole wiki syntax. The formal text is automatically included as formal Agora islands, and aligned with the corresponding informal snippets, allowing their simultaneous view.

A related project is the MMT [48] formal logical framework and theory browser, where different formal libraries together with their foundations are *deeply embedded* into the logic of the framework, allowing translations between them and their (at least theoretical) combined use within the framework. The associated HTML theory browser already includes versions of the Mizar [23] and HOL Light libraries [26], however, so far this technology rather serves interested readers than as an user-updatable collaborative wiki.

Another interesting project that combines statistical and semantic methods for aligning the terminologies and theorems in different formal libraries has been started recently [14]. This project shows that quite a lot of such alignment can be achieved fully automatically based on the structure of the library statements. Such tools could complement the manual alignment and annotations used by systems like Agora.

#### 4.4 Server-Based Assisting Tools: HOL(y)Hammer and MizAR

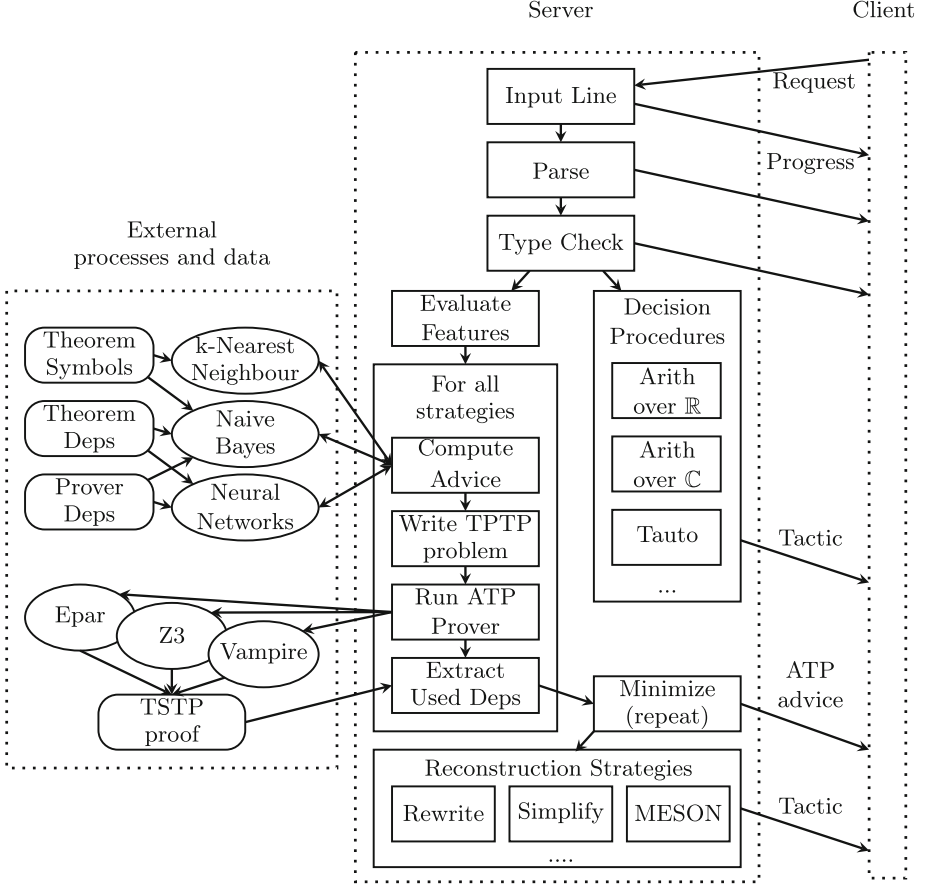
In general, collaboration with machines is a very interesting aspect of the fully formal domain. The large formal libraries can be subjected both to various data-mining and machine learning methods that are being developed for less semantic domains such as web search. On the other hand, one can also use very strong semantic search methods such as automated theorem proving (ATP), to assist the humans with finding proofs. Since the final correctness and consistency of such strong machine assistance is checked by the ITP systems (which are today very secure), one can use very efficiently implemented AI/ATP tools as parts of such toolchains, without a risk of introducing an incorrect proof due to implementational errors. The two main examples of such server-based assistance systems are the HOL(y)Hammer system serving the HOL Light users, and the MizAR system, serving the Mizar users. Since the two systems are otherwise quite similar, below we only explain their workings for the case of HOL(y)Hammer. The details of MizAR can be found in [27, 58, 59].

HOL(y)Hammer is an online AI/ATP service for formal (computer-understandable) mathematics encoded in the HOL Light system. The service allows its users to upload or modify and automatically process an arbitrary formal development (project) based on HOL Light, and to attack with automated theorem provers arbitrary conjectures that use the concepts defined in some of the uploaded projects. For that, the service uses several automated reasoning systems combined with several premise selection methods [36] trained on all the project proofs. The projects that are readily available on the server for such query answering include the recent versions of the Flyspeck, Multivariate Analysis and Complex Analysis libraries. The wiki-like features include upload and modification of existing projects, git-based versioning, HTML rendering of the libraries and their optional linking to Wikipedia and DBpedia and production of the OMDoc-based annotations for systems like Agora. The service runs on a 48-CPU server, currently employing in parallel for each task 7 AI/ATP combinations and 4 decision procedures that contribute to its overall performance. The current version of the system can prove about 40 % of all Flyspeck toplevel lemmas fully automatically, thus significantly speeding up the formalization efforts.

The overall problem solving architecture without the updating functions is shown in Fig. 2. The service receives a query (a HOL conjecture to prove, possibly with local assumptions) generated by one of the clients/frontends (Emacs, web interface, HOL session, etc.). The parsed query is processed in parallel by the (time-limited) AI/ATP combinations and the native HOL Light decision procedures (each managed by its forked HOL Light process, and terminated/killed by the master process if not finished within its global time limit). Each of the AI/ATP processes computes a specific feature representation of the query (used for knowledge selection), and sends such features to a specific instance of a premise advisor trained (using the particular feature representation) on previous proofs. Each of the advisors replies with a specific number of premises, which are then translated to a suitable ATP format, and written to a temporary file on which a specific ATP is run. The successful ATP result is then (pseudo-)minimized, and handed over to the combination of HOL Light proof-reconstruction procedures. These procedures again run in parallel, and if any of them is successful, the result is sent as a particular tactic application to the frontend. In case a native HOL Light decision procedure finds a proof, the result (again a particular tactic application) can be immediately sent to the frontend.

Since formal projects frequently modify their terminology and theorem names, and updating all the AI/ATP data on the server is expensive, HOL(y)Hammer has introduced *recursive content-based encoding* of the formal theorem and symbol names [56] for all projects and for re-using information across different projects and their versions. This is implemented as follows:

1. The name of every defined symbol is replaced by the content hash (we use MD5) of its variable-normalized definition containing the full types of the variables. This definition already uses content hashes instead of the previously defined symbols. This means that symbol names are no longer relevant in the whole project, neither white space and variable names.



**Fig. 2.** Overview of the HOL(y)Hammer problem solving functions

2. The name of each theorem is also replaced by the content hash of its (analogously normalized) statement.
3. The proof-dependency data extracted in the content encoding from all projects are copied to a special “common” directory.
4. Whenever a project  $P$  is started or modified, we find the intersection of the content-encoded names of the project’s theorems with such names that already exist in other projects/versions.
5. For each of such “already known” theorems  $T$  in  $P$ , we re-use for the AI/ATP systems its “already known” proofs  $D$  that are *compatible* with  $P$ ’s proof graph. This means, that the names of the proof dependencies of  $T$  in  $D$  must also exist in  $P$  (i.e., these theorems have been also proved in  $P$ , modulo the content-naming), and that these theorems precede  $T$  in  $P$  in its chronological order of library development (otherwise we might get cyclic training data for  $P$ ).

This procedure very significantly improves the live updates of the projects, because typically over 90 % of the library’s new version remains the same after these normalization steps [29].

## 5 Conclusions

The world of formal mathematical wikis is still very young, and it is even hard to say that there is a single most advanced system or that a particular wiki-like system has been largely adopted by the formal community. In this article we have therefore tried to summarize the topics that have emerged recently, when the venerable fifty-year old field of formalization of mathematics, with its venerable history of efforts such as QED, got interested in the collaborative wiki-like technologies.

Some of the pieces of the technology have already made it to the formal mainstream. Examples are advanced version control systems that have additional verification and rendering features. Other wiki-related techniques, such as web-based editing supported by verification in ProofWeb, or strong semantic advisors such as HOL(y)Hammer and MizAR have also already found some adopters. The various meta-wiki ideas and ideas linking the formal libraries with semantic web and informal mathematics are however still waiting to be fully developed and utilized.

There are many interesting topics that this brief overview has not dealt with. For example, change propagation is an important research topic and fine-grained dependency tracking can very significantly improve the speed of consistency checking of library updates [4]. Strong semantic assistance can include semi-automated translations between the informal and formal mathematics, with the outlook of gradually training smarter and smarter natural-language-understanding tools for mathematics on such aligned corpora [30]. And very interesting topics arise with future – hopefully wiki-supported – formalizations of other exact sciences such as physics. There, the many possible models of the (never completely known) real world need to somehow formally co-exist, while still being linked to accessible informal explanations about their relations to the underlying reality that they try to capture.

Perhaps the biggest challenge of formal mathematical wikis thus today seem to be the varied and evolving treatments of formality. While fully formal and computer-verified mathematics has made great steps forward in the last decade, there is still no agreement on the ultimate formalism. This includes the particular steps and stages that should link informal and formal knowledge and smoothly proceed in a wiki-like way from one to the other. On the other hand, formal mathematics and the strong semantic tools that have become available for its assistance are already clearly showing how disruptive for our way of doing science could be frameworks that achieve very smooth transition from informal (human-understandable) to formal (machine-understandable) scientific knowledge.

**Acknowledgments.** The following colleagues have collaborated with us on various aspects of formal wikis and on a number of formal wiki-related systems mentioned in

this paper: Mark Adams, Jesse Alama, Grzegorz Bancerek, Kasper Brink, Johan Commelin, Pierre Corbineau, Thibault Gauthier, Herman Geuvers, Mihnea Iancu, James McKinna, Michael Kohlhase, Christoph Lange, Lionel Mamane, Florian Rabe, Piotr Rudnicki, Geoff Sutcliffe, Carst Tankink, Jiri Vyskocil and Freek Wiedijk. Thanks to the anonymous SWCS referees for their valuable comments.

## References

1. The QED Manifesto. In: Bundy, A. (ed.) CADE 1994. LNCS, vol. 814, pp. 238–251. Springer, Heidelberg (1994)
2. Afshar, S.K., Siddique, U., Mahmoud, M.Y., Aravantinos, V., Seddiki, O., Hasan, O., Tahar, S.: Formal analysis of optical systems. *Math. Comput. Sci.* **8**(1), 39–70 (2014)
3. Alama, J., Brink, K., Mamane, L., Urban, J.: Large formal wikis: issues and solutions. In: Davenport, J.H., Farmer, W.M., Urban, J., Rabe, F. (eds.) MKM 2011 and Calculemus 2011. LNCS, vol. 6824, pp. 133–148. Springer, Heidelberg (2011)
4. Alama, J., Mamane, L., Urban, J.: Dependencies in formal mathematics: applications and extraction for Coq and Mizar. In: Jeuring et al. [24], pp. 1–16
5. Asperti, A., Ricciotti, W.: A web interface for Matita. In: Jeuring et al. [24], pp. 417–421
6. Aspinall, D.: Proof general: a generic tool for proof development. In: Graf, S., Schwartzbach, M. (eds.) TACAS 2000. LNCS, vol. 1785, pp. 38–43. Springer, Heidelberg (2000)
7. Autexier, S., Calmet, J., Delahaye, D., Ion, P.D.F., Rideau, L., Rioboo, R., Sexton, A.P. (eds.): AISC/Calculemus/MKM 2010. LNCS, vol. 6167. Springer, Heidelberg (2010)
8. Bancerek, G., Rudnicki, P.: A Compendium of Continuous Lattices in MIZAR. *J. Autom. Reasoning* **29**(3–4), 189–224 (2002)
9. Baumeister, J., Reutelshoefer, J., Puppe, F.: KnowWE: a semantic Wiki for knowledge engineering. *Appl. Intell.* **35**(3), 323–344 (2011)
10. Carette, J., Aspinall, D., Lange, C., Sojka, P., Windsteiger, W. (eds.): Intelligent Computer Mathematics. LNCS, vol. 7961. Springer, Heidelberg (2013)
11. The Coq Proof Assistant. <http://coq.inria.fr>
12. Corbineau, P., Kaliszyk, C.: Cooperative repositories for formal proofs. In: Kauers, M., Kerber, M., Miner, R., Windsteiger, W. (eds.) MKM/CALCULEMUS 2007. LNCS (LNAI), vol. 4573, pp. 221–234. Springer, Heidelberg (2007)
13. de Bruijn, N.G.: The mathematical language AUTOMATH, its usage, and some of its extensions. In: Laudet, M., Lacombe, D., Nolin, L., Schützenberger, M. (eds.) Symposium on Automatic Demonstration. LNM, vol. 125, pp. 29–61. Springer, Heidelberg (1968)
14. Gauthier, T., Kaliszyk, C.: Matching concepts across HOL libraries. In: Watt et al. [60], pp. 267–281
15. Gonthier, G.: The four colour theorem: engineering of a formal proof. In: Kapur, D. (ed.) ASCM 2007. LNCS (LNAI), vol. 5081, p. 333. Springer, Heidelberg (2008)
16. Gonthier, G., et al.: A machine-checked proof of the Odd Order Theorem. In: Blazy, S., Paulin-Mohring, C., Pichardie, D. (eds.) ITP 2013. LNCS, vol. 7998, pp. 163–179. Springer, Heidelberg (2013)
17. Grabowski, A., Kornilowicz, A., Naumowicz, A.: Mizar in a nutshell. *J. Formalized Reasoning* **3**(2), 153–245 (2010)

18. Hales, T.: Dense Sphere Packings: A Blueprint for Formal Proofs. LMS, vol. 400. Cambridge University Press, Cambridge (2012)
19. Hales, T.C., Adams, M., Bauer, G., Dang, D.T., Harrison, J., Hoang, t.L., Kaliszyk, C., Magron, V., McLaughlin, S., Nguyen, T.T., Nguyen, T.Q., Nipkow, T., Obua, S., Pleso, J., Rute, J., Solovyev, A., Ta, A.H.T., Tran, T.N., Trieu, D.T., Urban, J., Vu, K.K., Zumkeller, R.: A formal proof of the Kepler conjecture (2015). CoRR, abs/1501.02155
20. Harrison, J.: HOL Light: a tutorial introduction. In: Srivas, M., Camilleri, A. (eds.) FMCAD 1996. LNCS, vol. 1166, pp. 265–269. Springer, Heidelberg (1996)
21. Harrison, J., Urban, J., Wiedijk, F.: History of interactive theorem proving. In: Siekmann, J.H. (ed.) Computational Logic. Handbook of the History of Logic, vol. 9, pp. 135–214. North-Holland, Amsterdam (2014)
22. Hendriks, M., Kaliszyk, C., van Raamsdonk, F., Wiedijk, F.: Teaching logic using a state-of-the-art proof assistant. Acta Didactica Napocensia **3**(2), 35–48 (2010)
23. Iancu, M., Kohlhase, M., Rabe, F., Urban, J.: The Mizar mathematical library in OMDoc: Translation and applications. J. Autom. Reasoning **50**(2), 191–202 (2013)
24. Jeuring, J., Campbell, J.A., Carette, J., Dos Reis, G., Sojka, P., Wenzel, M., Sorge, V. (eds.): Intelligent Computer Mathematics. LNCS, vol. 7362. Springer, Heidelberg (2012)
25. Kaliszyk, C.: Web interfaces for proof assistants. In: Autexier, S., Benz Müller, C. (eds.) Proceedings of the Workshop on User Interfaces for Theorem Provers (UITP 2006), vol. 174, no. 2 of ENTCS, pp. 49–61 (2007)
26. Kaliszyk, C., Rabe, F.: Towards knowledge management for HOL Light. In: Watt et al. [60], pp. 357–372
27. Kaliszyk, C., Urban, J.: MizAR 40 for Mizar 40 (2013). CoRR, abs/1310.2805
28. Kaliszyk, C., Urban, J.: Learning-assisted automated reasoning with Flyspeck. J. Autom. Reasoning **53**(2), 173–213 (2014)
29. Kaliszyk, C., Urban, J.: HOL(y)Hammer: online ATP service for HOL Light. Math. Comput. Sci. **9**(1), 5–22 (2015)
30. Kaliszyk, C., Urban, J., Vyskocil, J., Geuvers, H.: Developing corpus-based translation methods between informal, formal mathematics: project description. In: Watt et al. [60], pp. 435–439
31. Kaliszyk, C., Wiedijk, F.: Merging procedural and declarative proof. In: Berardi, S., Damiani, F., de'Liguoro, U. (eds.) TYPES 2008. LNCS, vol. 5497, pp. 203–219. Springer, Heidelberg (2009)
32. Kaufmann, M., Moore, J.S.: An ACL2 tutorial. In: Mohamed et al. [43], pp. 17–21
33. Klein, G., Andronick, J., Elphinstone, K., Heiser, G., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., Winwood, S.: seL4: formal verification of an operating-system kernel. Commun. ACM **53**(6), 107–115 (2010)
34. Kohlhase, M.: OMDoc 2006. LNCS, vol. 4180. Springer, Heidelberg (2006)
35. Kühlwein, D., Blanchette, J.C., Kaliszyk, C., Urban, J.: MaSh: machine learning for Sledgehammer. In: Blazy, S., Paulin-Mohring, C., Pichardie, D. (eds.) ITP 2013. LNCS, vol. 7998, pp. 35–50. Springer, Heidelberg (2013)
36. Kühlwein, D., van Laarhoven, T., Tsvitshivadze, E., Urban, J., Heskes, T.: Overview and evaluation of premise selection techniques for large theory mathematics. In: Gramlich, B., Miller, D., Sattler, U. (eds.) IJCAR 2012. LNCS, vol. 7364, pp. 378–392. Springer, Heidelberg (2012)
37. Lange, C.: OMDoc ontology (2011). <http://kwarc.info/projects/docOnto/omdoc.html>

38. Lange, C.: Ontologies and languages for representing mathematical knowledge on the semantic web. *Semantic Web* **4**(2), 119–158 (2013)
39. Lange, C., Rowat, C., Kerber, M.: The ForMaRE project - formal mathematical reasoning in economics. In: Carette et al. [10], pp. 330–334
40. Lange, C., Urban, J. (eds.): Proceedings of the ITP Workshop on Mathematical Wikis (MathWikis), no. 767 in CEUR Workshop Proceedings, Aachen (2011)
41. Leroy, X.: Formal verification of a realistic compiler. *Commun. ACM* **52**(7), 107–115 (2009)
42. Matuszewski, R. (ed.): The QED Workshop II, Warsaw University Technical report No. L/1/95 (1995)
43. Mohamed, O.A., Muñoz, C., Tahar, S. (eds.): Theorem Proving in Higher Order Logics. LNCS, vol. 5170. Springer, Heidelberg (2008)
44. Nalepa, G.J.: Collective knowledge engineering with semantic wikis. *J. UCS* **16**(7), 1006–1023 (2010)
45. Niles, I., Pease, A.: Towards a standard upper ontology. In: FOIS, pp. 2–9 (2001)
46. Owre, S., Shankar, N.: A brief overview of PVS. In: Mohamed et al. [43], pp. 22–27
47. Pottier, L.: LogiCoq (1999). URL: <http://wims.unice.fr/wims/wims.cgi?module=U3/logic/logicoq>
48. Rabe, F.: The MMT API: a generic MKM system. In: Carette et al. [10], pp. 339–343
49. Ramachandran, D., Reagan, P., Goolsbey, K.: First-orderized ResearchCyc: expressiveness and efficiency in a common sense knowledge base. In: Shvaiko P. (ed.) Proceedings of the Workshop on Contexts and Ontologies: Theory, Practice and Applications (2005)
50. Ring, M., Lüth, C.: Collaborative interactive theorem proving with clide. In: Klein, G., Gamboa, R. (eds.) ITP 2014. LNCS, vol. 8558, pp. 467–482. Springer, Heidelberg (2014)
51. Robinson, J.A., Voronkov, A. (eds.): Handbook of Automated Reasoning (in 2 volumes). Elsevier and MIT Press, New York (2001)
52. Schürmann, C.: The Twelf proof assistant. In: Berghofer, S., Nipkow, T., Urban, C., Wenzel, M. (eds.) TPHOLs 2009. LNCS, vol. 5674, pp. 79–83. Springer, Heidelberg (2009)
53. Tankink, C., Kaliszyk, C., Urban, J., Geuvers, H.: Formal mathematics on display: a wiki for Flyspeck. In: Carette et al. [10], pp. 152–167
54. Tankink, C., Lange, C., Urban, J.: Point-and-write - documenting formal mathematics by reference. In: Jeuring et al. [24], pp. 169–185
55. Urban, J.: XML-izing Mizar: making semantic processing and presentation of MML easy. In: Kohlhase, M. (ed.) MKM 2005. LNCS (LNAI), vol. 3863, pp. 346–360. Springer, Heidelberg (2006)
56. Urban, J.: Content-based encoding of mathematical and code libraries. In: Lange, C., Urban, J. (eds.) Proceedings of the ITP Workshop on Mathematical Wikis (MathWikis), no. 767 in CEUR Workshop Proceedings, pp. 49–53, Aachen (2011)
57. Urban, J., Alama, J., Rudnicki, P., Herman Geuvers, A.: Wiki for Mizar: Motivation, considerations, and initial prototype. In: Autexier et al. [7], pp. 455–469
58. Urban, J., Rudnicki, P., Sutcliffe, G.: ATP and presentation service for Mizar formalizations. *J. Autom. Reasoning* **50**, 229–241 (2013)
59. Urban, J., Sutcliffe, G.: Automated reasoning and presentation support for formalizing mathematics in Mizar. In: Autexier et al. [7], pp. 132–146
60. Watt, S.M., Davenport, J.H., Sexton, A.P., Sojka, P., Urban, J. (eds.): CICM 2014. LNCS, vol. 8543. Springer, Heidelberg (2014)

61. Wenzel, M., Paulson, L.C., Nipkow, T.: The Isabelle framework. In: Mohamed et al. [43], pp. 33–38
62. Worden, L.: WorkingWiki: a MediaWiki-based platform for collaborative research. In: Lange and Urban [40], pp. 63–73